Experiment 6

Student Name: Gourav Sharma **UID: 23BCS10857**
Branch: CSE **Section/Group: KRG 3-A**
Semester: 6th **Date of Performance:26/02/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**:To design a YouTube-like video streaming platform architecture using draw.io that supports HLS/DASH based adaptive streaming with proper manifest file usage, enables both static (VOD) and live streaming, and implements likes and comments handling without using a pub-sub system like Kafka, including support for static comments via HTTP APIs and live comments via WebSocket-based real-time updates.

2. **Objective**:

- To design a scalable architecture for a YouTube-like video streaming platform.
- To implement HLS/DASH adaptive streaming using manifest files.
- To support both Video on Demand (VOD) and live streaming in the system.
- To implement likes and comments handling without using a pub-sub system like Kafka.
- To enable real-time interaction using WebSocket for live comments and engagement.

3. **Tools Used:**

- **Frontend (ReactJS, HTML, CSS, JavaScript, WebSocket)**
  - Built responsive UI with real-time communication support
- **Backend (Node.js / Java / Spring Boot)**
  - Developed scalable REST APIs & WebSocket server
- **Database (MongoDB / MySQL / PostgreSQL)**
  - Managed structured & unstructured data efficiently
- **Redis**
  - Implemented caching & pub-sub for high performance
- **Deployment (AWS / GCP, Load Balancer, Docker)**
  - Cloud deployment with scalability, containerization & traffic distribution

4. **System Requirements:**

A. Functional Requirements

- Users should be able to create an account and login to the platform.
- Content creators should be able to upload videos to the platform.
- The system should support video streaming using HLS or DASH with manifest files (.m3u8 / .mpd).

- Users should be able to watch videos using adaptive bitrate streaming based on network bandwidth.
- Users should be able to like and comment on videos.
- The platform should support real-time likes and live comments using WebSocket (without using Kafka/pub-sub).
- Users should be able to view static comments fetched through HTTP APIs.
- The system should support live streaming where streamers send video through RTMP ingest servers.
- Users should be able to search videos based on title, creator name, or tags.

B. Non-Functional Requirements

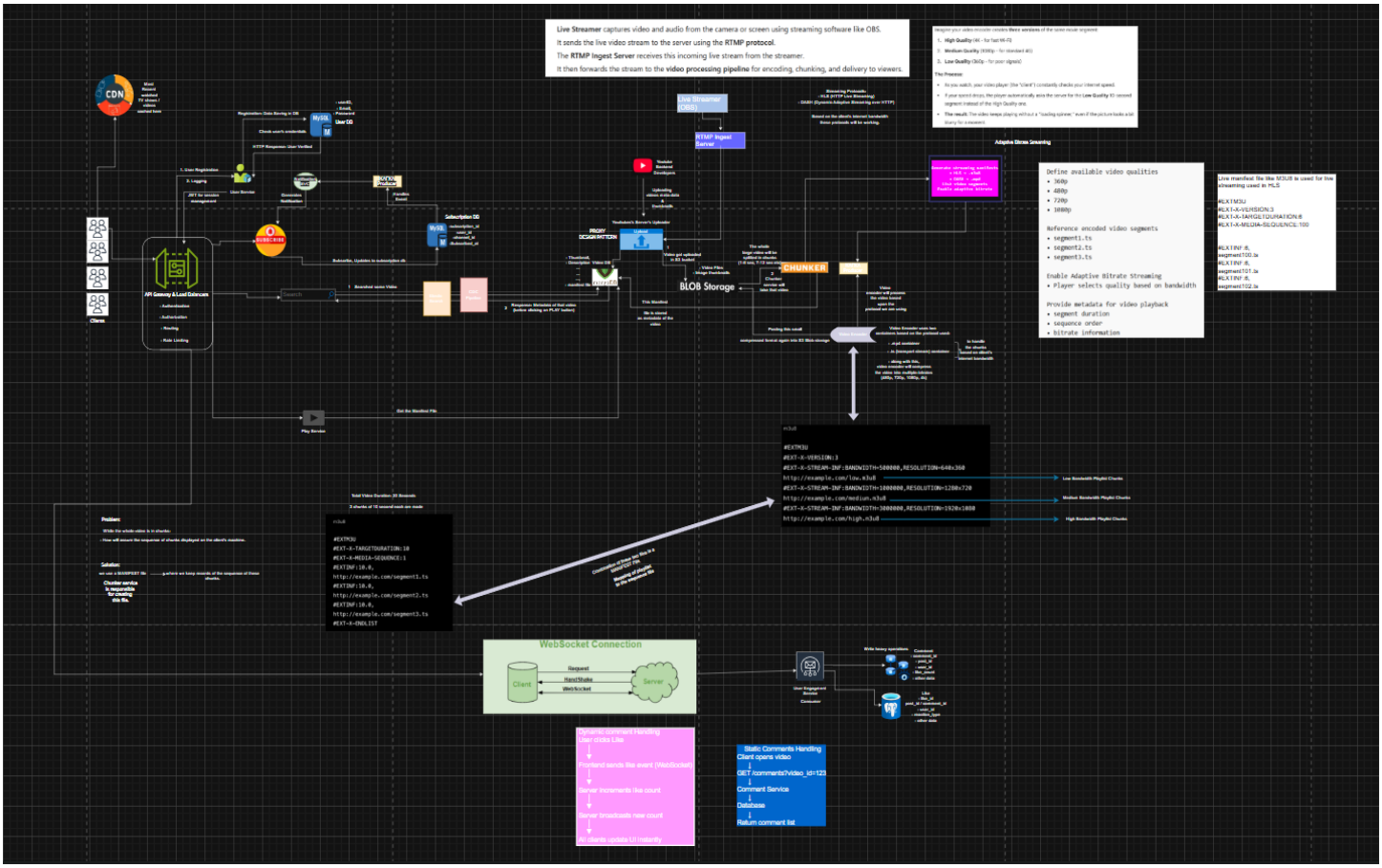
- **Scalability**
  The system should support millions of users simultaneously streaming videos and uploading content, while handling a large library of videos.

- **CAP Theorem**
  Availability >>> Consistency
  The platform should ensure high availability so that users can watch videos or live streams anytime with minimal downtime.

- **Latency:**
  Video playback latency should be **low (around 50–80 ms)** with minimal buffering during streaming.

- The system should deliver videos efficiently using HLS/DASH adaptive streaming and CDN distribution to maintain smooth playback across different network conditions.

5. Low Level Design (LLD):



6. Scalability Solution

- Use CDN (Content Delivery Network) to distribute video content globally.
- Store video segments in distributed object storage (Blob/S3).
- Use video encoding and chunking to enable adaptive bitrate streaming.
- Separate services for video processing, engagement (likes/comments), and streaming.
- Use load balancers and horizontal scaling to handle millions of users.

7. Learning Outcomes (What I Have Learnt)

- Understand the architecture of large-scale video streaming platforms.
- Learn how HLS/DASH streaming and manifest files work.
- Gain knowledge about handling real-time features using WebSockets.
- Understand the difference between live streaming and static video streaming.
- Learn how to design scalable backend systems for media platforms.

.